

Programação Orientada a Objetos (POO) em Java

Um guia conceitual e prático com os pilares fundamentais da engenharia de software

1. Introdução à Programação Orientada a Objetos

A Programação Orientada a Objetos (POO) é um paradigma de desenvolvimento de software que modela o mundo real por meio de estruturas chamadas **Objetos**. Ao contrário da programação estruturada, que se concentra em funções e lógica linear, a POO organiza o código em torno de dados (atributos) e comportamentos (métodos).

O Java é uma linguagem nativamente orientada a objetos. Exceto pelos tipos primitivos (como `int`, `char`, `boolean`), praticamente todo o código Java reside dentro de classes e interage através de objetos.

Classe vs. Objeto

Pense na **Classe** como a planta arquitetônica (o molde) de uma casa, e no **Objeto** como a casa física construída a partir dessa planta. É possível construir inúmeras casas (objetos) idênticas ou personalizadas utilizando a mesma planta (classe).

2. Os Quatro Pilares da POO

A arquitetura de sistemas robustos e escaláveis em Java sustenta-se sobre quatro pilares fundamentais: **Abstração**, **Encapsulamento**, **Herança** e **Polimorfismo**.

2.1. Abstração

A abstração consiste em isolar e extrair apenas as características e comportamentos essenciais de um objeto do mundo real que são relevantes para o sistema, ignorando os detalhes complexos ou desnecessários.

Em Java, implementamos a abstração por meio de `abstract class` (classes abstratas) e `interface`.

```
// Definição de uma classe abstrata que serve de molde para sub-classes
public abstract class ContaBancaria {
    private String titular;
    protected double saldo;

    public ContaBancaria(String titular) {
        this.titular = titular;
    }

    // Método abstrato: não possui implementação na classe pai
    public abstract void aplicarTaxaMensal();

    // Método concreto
    public void depositar(double valor) {
        if (valor > 0) { this.saldo += valor; }
    }
}
```

2.2. Encapsulamento

O encapsulamento protege o estado interno de um objeto ocultando os seus atributos diretamente e expondo o acesso apenas através de métodos públicos seguros (geralmente métodos *Getters* e *Setters*). Isso evita alterações acidentais e mantém a consistência dos dados (regras de negócio).

Modificador	Mesma Classe	Mesmo Pacote	Sub-classes (Herança)	Todos (Global)
private	Sim	Não	Não	Não
default (padrão)	Sim	Sim	Não	Não
protected	Sim	Sim	Sim	Não
public	Sim	Sim	Sim	Sim

```

public class Produto {
    // Atributo privado impede o acesso direto externo
    private double preco;

    // Getter para leitura controlada
    public double getPreco() { return this.preco; }

    // Setter com validação de consistência (Regra de Negócio)
    public void setPreco(double preco) {
        if (preco >= 0) {
            this.preco = preco;
        } else {
            System.out.println("Preço inválido!");
        }
    }
}

```

2.3. Herança

A herança permite que uma nova classe (sub-classe ou classe filha) herde atributos e métodos de uma classe existente (super-classe ou classe pai). Em Java, utiliza-se a palavra-chave `extends`. Isso promove o reaproveitamento de código e estabelece uma relação do tipo "é um".

```

// Classe Pai
public class Funcionario {
    protected String nome;
    protected double salarioBase;

    public double calcularSalario() { return this.salarioBase; }
}

// Classe Filha herda tudo de Funcionario
public class Gerente extends Funcionario {
    private double bonus;

    // Sobrescrita do método da super-classe
    @Override
    public double calcularSalario() {
        return super.calcularSalario() + this.bonus;
    }
}

```

2.4. Polimorfismo

Polimorfismo significa "muitas formas". É a capacidade de um mesmo identificador (como a chamada de um método) invocar comportamentos diferentes dependendo do objeto real que o está a executar.

- **Sobrescrita (Override):** Ocorre em tempo de execução (dinâmico). A classe filha reescreve a lógica de um método da super-classe.
- **Sobrecarga (Overload):** Ocorre em tempo de compilação (estático). A mesma classe possui métodos com o mesmo nome, mas assinaturas (parâmetros) diferentes.

```
// Demonstração prática de Polimorfismo Dinâmico
public class CalculadorDeFolha {
    // O método aceita qualquer objeto que "seja um" Funcionario
    public void registrarPagamento(Funcionario f) {
        // O comportamento muda se f for um Gerente ou um Diretor
        double total = f.calcularSalario();
        System.out.println("Pagando o valor de: €" + total);
    }
}
```

3. Interfaces vs. Classes Abstratas

A escolha entre usar interfaces ou classes abstratas é crucial no design de arquiteturas Java:

- **Classes Abstratas:** Podem conter estado (atributos não estáticos) e comportamento parcial (métodos prontos). Use quando houver um vínculo conceitual forte de herança ("é um"). O Java permite estender apenas *uma* classe pai.
- **Interfaces:** Definem um contrato puramente comportamental. Não guardam estados de instância. Use para estabelecer papéis comuns entre classes de origens totalmente distintas. Uma classe em Java pode implementar *múltiplas* interfaces.

4. Conclusão e Boas Práticas

O uso correto da POO em Java confere ao código propriedades fundamentais:

Mabilidade = Coesão + Baixo Acoplamento

Ao construir seus sistemas para o portfólio ou ambiente corporativo, lembre-se sempre de manter as classes com responsabilidade única (o 'S' do SOLID) e proteger os atributos com o correto nível de encapsulamento. Isso garante códigos fáceis de testar, manter e expandir.